

Q1. Implement the OR and AND problem using perceptron learning model.

```
print("OR PROBLEM")
def g(x):
    if x > 0 :
        return 1
    else : return 0
inputs = [[0,0], [0,1], [1,0], [1,1]]
outputs = [0, 1, 1, 1]
weights = [0.5, 0, 1]
n = 0.25
old_weights = []
iterations = 0
while old_weights!=weights:
    iterations+=1
    old_weights = weights.copy()
    for i in range(4):
        inp, op = inputs[i], outputs[i]
        x, y = inp[0], inp[1]
        val = g(weights[0] + weights[1]*x + weights[2]*y)
        if (val!=op):
            weights[0] = weights[0] + n*(op - val)*1
            weights[1] = weights[1] + n*(op - val)*x
            weights[2] = weights[2] + n*(op - val)*y
    print(weights, '\t', val)
    print(old_weights, '\t', weights, '\n')
print("\nNumber of Iterations are : ", iterations)
```

OR PROBLEM

[0.25, 0.0, 1.0]	1
[0.25, 0.0, 1.0]	1
[0.25, 0.0, 1.0]	1
[0.25, 0.0, 1.0]	1
[0.5, 0, 1]	[0.25, 0.0, 1.0]

[0.0, 0.0, 1.0]	1
[0.0, 0.0, 1.0]	1
[0.25, 0.25, 1.0]	0
[0.25, 0.25, 1.0]	1
[0.25, 0.0, 1.0]	[0.25, 0.25, 1.0]

[0.0, 0.25, 1.0]	1
[0.0, 0.25, 1.0]	1
[0.0, 0.25, 1.0]	1
[0.0, 0.25, 1.0]	1
[0.25, 0.25, 1.0]	[0.0, 0.25, 1.0]

[0.0, 0.25, 1.0]	0
[0.0, 0.25, 1.0]	1

[0.0, 0.25, 1.0]	1
[0.0, 0.25, 1.0]	1
[0.0, 0.25, 1.0]	[0.0, 0.25, 1.0]

Number of Iterations are : 4

```

print("AND PROBLEM")
inputs = [[0,0], [0,1], [1,0], [1,1]]
outputs = [0, 0, 0, 1]
weights = [0.5, 0, 1]
n = 0.25
old_weights = []
iterations = 0
while old_weights!=weights:
    iterations+=1
    old_weights = weights.copy()
    for i in range(4):
        inp, op = inputs[i], outputs[i]
        x, y = inp[0], inp[1]
        val = g(weights[0] + weights[1]*x + weights[2]*y)
        if (val!=op):
            weights[0] = weights[0] + n*(op - val)*1
            weights[1] = weights[1] + n*(op - val)*x
            weights[2] = weights[2] + n*(op - val)*y
    print(weights, '\t', val)
    print(old_weights, '\t', weights, '\n')
print("\nNumber of Iterations are : ", iterations)

```

AND PROBLEM

[0.25, 0.0, 1.0]	1
[0.0, 0.0, 0.75]	1
[0.0, 0.0, 0.75]	0
[0.0, 0.0, 0.75]	1
[0.5, 0, 1]	[0.0, 0.0, 0.75]

[0.0, 0.0, 0.75]	0
[-0.25, 0.0, 0.5]	1
[-0.25, 0.0, 0.5]	0
[-0.25, 0.0, 0.5]	1
[0.0, 0.0, 0.75]	[-0.25, 0.0, 0.5]

[-0.25, 0.0, 0.5]	0
[-0.5, 0.0, 0.25]	1
[-0.5, 0.0, 0.25]	0
[-0.25, 0.25, 0.5]	0
[-0.25, 0.0, 0.5]	[-0.25, 0.25, 0.5]

[-0.25, 0.25, 0.5]	0
[-0.5, 0.25, 0.25]	1

```

[-0.5, 0.25, 0.25]    0
[-0.25, 0.5, 0.5]    0
[-0.25, 0.25, 0.5]    [-0.25, 0.5, 0.5]

[-0.25, 0.5, 0.5]    0
[-0.5, 0.5, 0.25]    1
[-0.5, 0.5, 0.25]    0
[-0.5, 0.5, 0.25]    1
[-0.25, 0.5, 0.5]    [-0.5, 0.5, 0.25]

[-0.5, 0.5, 0.25]    0
[-0.5, 0.5, 0.25]    0
[-0.5, 0.5, 0.25]    0
[-0.5, 0.5, 0.25]    1
[-0.5, 0.5, 0.25]    [-0.5, 0.5, 0.25]

```

Number of Iterations are : 6

Q2. Implement same with vector notation where: $y=g(WTX)$ and $WT=WT(old)+\alpha(t-y).XT$

```

import numpy as np
weights = np.array(((0.5, -0.5), (-0.5, 0), (0.4, -0.2), (-0.1, -0.1)))
X = np.array(((1, ), (2, ), (1, ), (3, )))
output = (((1, ), (0, )))
n = 0.3
old_weights = np.array(())
activate = np.vectorize(g)
iterations = 1
print(weights.T, '\n')
while old_weights.all() != weights.all():
    iterations += 1
    old_weights = weights.copy()
    y = activate(np.dot(weights.T, X))
    weights = weights + (n * (output - y) * X.T).T
    print(weights.T, '\n')
print("Number of iterations : ", iterations)

[[ 0.5 -0.5  0.4 -0.1]
 [-0.5  0.  -0.2 -0.1]]

[[ 0.8  0.1  0.7  0.8]
 [-0.5  0.  -0.2 -0.1]]

Number of iterations : 2

```

Q3. Download the Fashion MNIST dataset from <https://github.com/zalandoresearch/fashion-mnist> and develop an image classification

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, datasets, models
(train_images, train_labels), (test_images, test_labels) =
datasets.fashion_mnist.load_data()
train_images = train_images / 255.0
test_images = test_images / 255.0
train_images = train_images.reshape((train_images.shape[0], 28, 28,
1))
test_images = test_images.reshape((test_images.shape[0], 28, 28, 1))
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28,
1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_images, train_labels, epochs=5,
          validation_data=(test_images, test_labels))
history=model.fit(train_images, train_labels, epochs=5,
                  validation_data=(test_images, test_labels))
plt.plot((history.history['accuracy']))
plt.plot((history.history['val_accuracy']))
plt.plot((history.history['loss']))
plt.plot((history.history['val_loss']))
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['accuracy', 'val_accuracy', 'loss', 'val_loss'], loc='upper
left')
plt.show()
test_loss, test_accuracy = model.evaluate(test_images, test_labels)
print(f'\nTest accuracy: {test_accuracy:.4f}')
model.summary()

```

C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

    super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Epoch 1/5
1875/1875 _____ 61s 29ms/step - accuracy: 0.7458 -
loss: 0.6877 - val_accuracy: 0.8545 - val_loss: 0.3944
Epoch 2/5
1875/1875 _____ 46s 25ms/step - accuracy: 0.8770 -
loss: 0.3343 - val_accuracy: 0.8769 - val_loss: 0.3291
Epoch 3/5
1875/1875 _____ 48s 26ms/step - accuracy: 0.8957 -
loss: 0.2811 - val_accuracy: 0.8922 - val_loss: 0.2931
Epoch 4/5
1875/1875 _____ 47s 25ms/step - accuracy: 0.9088 -
loss: 0.2489 - val_accuracy: 0.8914 - val_loss: 0.2985
Epoch 5/5
1875/1875 _____ 43s 23ms/step - accuracy: 0.9176 -
loss: 0.2213 - val_accuracy: 0.8939 - val_loss: 0.2996
Epoch 1/5
1875/1875 _____ 13s 7ms/step - accuracy: 0.9258 - loss:
0.2017 - val_accuracy: 0.8983 - val_loss: 0.2870
Epoch 2/5
1875/1875 _____ 20s 6ms/step - accuracy: 0.9313 - loss:
0.1847 - val_accuracy: 0.9035 - val_loss: 0.2891
Epoch 3/5
1875/1875 _____ 12s 6ms/step - accuracy: 0.9393 - loss:
0.1665 - val_accuracy: 0.9055 - val_loss: 0.2755
Epoch 4/5
1875/1875 _____ 12s 7ms/step - accuracy: 0.9438 - loss:
0.1477 - val_accuracy: 0.9062 - val_loss: 0.2757
Epoch 5/5
336/1875 _____ 10s 7ms/step - accuracy: 0.9496 - loss:
0.1357

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
predictions = model.predict(test_images)
predicted_labels = np.argmax(predictions, axis=1)
accuracy = accuracy_score(test_labels, predicted_labels)
print(f'Test accuracy: {accuracy:.4f}')
print(classification_report(test_labels, predicted_labels))
conf_matrix = confusion_matrix(test_labels, predicted_labels)
print('Confusion Matrix:')
print(conf_matrix)

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
plt.figure(figsize=(10,10))
for i in range(25):

```

```

plt.subplot(5,5,i+1)
plt.xticks([])
plt.yticks([])
plt.grid(False)
plt.imshow(train_images[i], cmap=plt.cm.binary)
plt.xlabel(class_names[train_labels[i]])
plt.show()

def plot_image(i, predictions_array, true_label, img):
    true_label, img = true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(img, cmap=plt.cm.binary)
    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'
    plt.xlabel("{} {:2.0f}% ({})" .format(class_names[predicted_label],
                                         100*np.max(predictions_array),
                                         class_names[true_label]),
              color=color)

def plot_value_array(i, predictions_array, true_label):
    true_label = true_label[i]
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')

num_rows = 5
num_cols = 3
num_images = num_rows*num_cols
plt.figure(figsize=(2*2*num_cols, 2*2*num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2*num_cols, 2*i+1)
    plot_image(i, predictions[i], test_labels, test_images)
    plt.subplot(num_rows, 2*num_cols, 2*i+2)
    plot_value_array(i, predictions[i], test_labels)
plt.tight_layout()
plt.show()

```